```
import os, json, re, io, anthropic, pickle, base64, tempfile, urllib.request, url
from flask import Flask, request, jsonify, send_file, render_template, session, r
from werkzeug.utils import secure_filename
import pdfplumber, openpyxl
from openpyxl.styles import Font, PatternFill, Alignment, Border, Side
from openpyxl.utils import get_column_letter
from datetime import datetime
from collections import defaultdict

app = Flask(__name__)
app.secret_key = os.environ.get('SECRET_KEY', 'mca-analyzer-secret-2026')
app.config['UPLOAD_FOLDER'] = os.environ.get('UPLOAD_FOLDER', '/tmp/uploads')
app.config['HISTORY_FOLDER'] = os.environ.get('HISTORY_FOLDER', '/tmp/history')
app.config['MAX_CONTENT_LENGTH'] = 32 * 1024 * 1024
ALLOWED_EXTENSIONS = {'pdf', 'csv', 'txt'}
os.makedirs(app.config['UPLOAD_FOLDER'], exist_ok=True)
os.makedirs(app.config['HISTORY_FOLDER'], exist_ok=True)

USERS = {
    os.environ.get('USERNAME1', 'dave'): os.environ.get('PASSWORD1', 'mca2026'),
    os.environ.get('USERNAME2', 'admin'): os.environ.get('PASSWORD2', 'analyze202
}

def login_required(f):
    from functools import wraps
    @wraps(f)
    def decorated(*args, **kwargs):
        if not session.get('logged_in'):
            return redirect(url_for('login'))
        return f(*args, **kwargs)
    return decorated

def allowed_file(f): return '.' in f and f.rsplit('.',1)[1].lower() in ALLOWED_EX

def extract_text_from_pdf(path):
    text_parts = []
    try:
        with pdfplumber.open(path) as pdf:
            total = len(pdf.pages)
            if total <= 12:
                pages_to_read = list(range(total))
            else:
                pages_to_read = list(range(8)) + list(range(max(8, total-4), tota
            for i in pages_to_read:
```

```
                try:
                    t = pdf.pages[i].extract_text()
                    if t:
                        text_parts.append(t[:3000])
                except:
                    pass
    except:
        return ""
    return "\n".join(text_parts)

def extract_text(path):
    ext = path.rsplit('.',1)[1].lower()
    if ext == 'pdf': return extract_text_from_pdf(path)
    with open(path,'r',encoding='utf-8',errors='ignore') as f: return f.read()

def calc_monthly(amount, frequency):
    freq = (frequency or 'weekly').lower()
    if 'daily' in freq: return amount * 22
    if 'bi' in freq: return amount * 2
    if 'monthly' in freq: return amount * 1
    return amount * 4

def save_history(company_name, data, excel_bytes):
    safe = re.sub(r'[^\w\s-]','',company_name).strip().replace(' ','_')
    ts = datetime.now().strftime('%Y%m%d_%H%M%S')
    entry_id = "{}_{}".format(safe, ts)
    path = os.path.join(app.config['HISTORY_FOLDER'], entry_id + '.pkl')
    with open(path, 'wb') as f:
        pickle.dump({'id':entry_id,'company_name':company_name,'data':data,'excel
    return entry_id

def load_history():
    entries = []
    folder = app.config['HISTORY_FOLDER']
    for fname in sorted(os.listdir(folder), reverse=True)[:10]:
        if fname.endswith('.pkl'):
            try:
                with open(os.path.join(folder, fname), 'rb') as f:
                    e = pickle.load(f)
                    entries.append({'id':e['id'],'company_name':e['company_name']
            except: pass
    return entries

def sanitize_data(obj):
    if isinstance(obj, dict):
        return {k: sanitize_data(v) for k, v in obj.items()}
    elif isinstance(obj, list):
```

```
        return [sanitize_data(i) for i in obj]
    elif isinstance(obj, str):
        result = []
        for ch in obj:
            cp = ord(ch)
            if cp < 0x20 and cp not in (0x09, 0x0a, 0x0d): continue
            if 0xD800 <= cp <= 0xDFFF: continue
            if 0xFFFE <= cp <= 0xFFFF: continue
            result.append(ch)
        s = ''.join(result)
        s = s.replace('\u2013','-').replace('\u2014','-')
        s = s.replace('\u2018',"'").replace('\u2019","'")
        s = s.replace('\u201c','"').replace('\u201d','"')
        s = s.replace('\u2022','*').replace('\u00a0',' ')
        s = s.replace('\u2026','...').replace('\u2212','-')
        s = s.replace('\u00b7','*').replace('\u25cf','*')
        return s
    return obj

def load_entry(entry_id):
    path = os.path.join(app.config['HISTORY_FOLDER'], entry_id + '.pkl')
    if not os.path.exists(path): return None
    with open(path, 'rb') as f:
        entry = pickle.load(f)
    if 'data' in entry:
        entry['data'] = sanitize_data(entry['data'])
    return entry

# ============================================================
# VALIDATION LAYER 1: MCA Pattern Rules Engine
# Deterministic rules — no AI guessing needed for these
# ============================================================
def run_rules_engine(raw_text):
    """
    Reads every ACH debit transaction individually.
    - Groups multiple loans from the same company into one combined position.
    - Detects stopped payments, amount changes, and completed loans.
    - No pattern assumptions — every transaction line is read directly.
    """
    # Parse every ACH debit line individually
    pattern = re.compile(
        r'(\d{1,2}/\d{1,2})\s+<?\s*(?:Business to Business ACH Debit|ACH Debit)\s
        r'([A-Za-z0-9 &./\-]+?)\s+(?:\S+\s+)*?([\d,]+\.\d{2})',
        re.IGNORECASE
    )

    # raw_transactions: list of (date, payee_raw, amount)
```

```
raw_transactions = []
for m in pattern.finditer(raw_text):
    date_str = m.group(1).strip()
    payee_raw = re.sub(r'\s+', ' ', m.group(2).strip())
    try:
        amt = float(m.group(3).replace(',',''))
    except:
        continue
    if amt <= 0:
        continue
    raw_transactions.append((date_str, payee_raw, amt))

if not raw_transactions:
    return []

# Normalize company name — strip account numbers, reference codes, dates
def normalize_company(name):
    # Remove trailing codes like "Cs1507", "xxxxx1234", "260204", "#28"
    name = re.sub(r'\s+[A-Z0-9#]{4,}\s*$', '', name, flags=re.IGNORECASE)
    name = re.sub(r'\s+\d{6}\s*$', '', name)
    name = re.sub(r'\s+x{3,}\d+\s*$', '', name, flags=re.IGNORECASE)
    # Take first 3 meaningful words as the company key
    words = name.strip().split()
    return ' '.join(words[:3]).lower()

# Group by normalized company name — combines multiple loans from same lender
by_company = defaultdict(list)
for date_str, payee_raw, amt in raw_transactions:
    key = normalize_company(payee_raw)
    by_company[key].append({
        'date': date_str,
        'payee_raw': payee_raw,
        'amount': amt
    })

# Determine sort order for dates (month/day strings)
def date_sort_key(d):
    try:
        parts = d.split('/')
        return int(parts[0]) * 100 + int(parts[1])
    except:
        return 0

positions = []
for company_key, txns in by_company.items():
    if len(txns) < 1:
        continue
```

```
# Sort transactions chronologically
txns.sort(key=lambda x: date_sort_key(x['date']))

# Find unique amounts for this company
amounts = [t['amount'] for t in txns]
unique_amounts = sorted(set(amounts))
most_recent_amt = txns[-1]['amount']
first_date = txns[0]['date']
last_date = txns[-1]['date']
count = len(txns)

# Check if multiple distinct loan amounts (same company, multiple loans)
# e.g. OnDeck has $3,544.69 AND $2,100.00 debiting separately
# Sum them if they appear in the same time window
if len(unique_amounts) > 1:
    # Could be amount change OR multiple loans
    # If different amounts appear on the SAME dates → multiple loans (com
    # If earlier amounts then later amounts → amount changed
    dates_per_amount = defaultdict(set)
    for t in txns:
        dates_per_amount[t['amount']].add(t['date'])

    # Check overlap — if two amounts share dates, they are concurrent loa
    amount_list = list(dates_per_amount.keys())
    concurrent = False
    if len(amount_list) >= 2:
        dates_a = dates_per_amount[amount_list[0]]
        dates_b = dates_per_amount[amount_list[1]]
        if dates_a & dates_b:  # overlap in dates
            concurrent = True

    if concurrent:
        # Multiple concurrent loans — combine into one position
        combined_amt = sum(unique_amounts)
        # Use most common payee name as display name
        display_name = max(set(t['payee_raw'] for t in txns),
                           key=lambda n: sum(1 for t in txns if t['payee_
        display_name = display_name[:35]
        loans_detail = ' + '.join(['${:,.2f}'.format(a) for a in sorted(u
        notes = 'Multiple loans combined: {} = ${:,.2f} per cycle'.format
        most_recent_amt = combined_amt
    else:
        # Amount changed over time
        old_amt = txns[0]['amount']
        new_amt = txns[-1]['amount']
        change_txn = next((t for t in txns if t['amount'] != old_amt), No
```

```
        change_date = change_txn['date'] if change_txn else '?'
        display_name = txns[-1]['payee_raw'][:35]
        notes = 'Amount changed from ${:,.2f} to ${:,.2f} on {}'.format(
            old_amt, new_amt, change_date)
else:
    display_name = txns[-1]['payee_raw'][:35]
    notes = ''

# Detect stopped payments — look for gap at end
# If the most recent date is significantly earlier than the end of statem
# we flag it, but only if we have enough months of data
all_dates = [t['date'] for t in txns]
last_month = max(date_sort_key(d) for d in all_dates)
# Find the latest month in the entire raw text
all_statement_dates = re.findall(r'\b(\d{1,2}/\d{1,2})\b', raw_text)
if all_statement_dates:
    latest_statement = max(date_sort_key(d) for d in all_statement_dates)
    # If last payment was more than 5 weeks before latest statement date
    last_month_num = last_month // 100
    last_day_num = last_month % 100
    latest_month_num = latest_statement // 100
    months_gap = latest_month_num - last_month_num
    if months_gap >= 1 and count >= 2:
        if 'changed' not in notes.lower():
            notes = ('stopped after {}'.format(last_date) +
                     (' — possibly paid off' if count >= 4 else ' — verif
                     ('; ' + notes if notes else ''))

# Determine frequency by analyzing gaps between dates
# For combined loans, analyze gaps using only one loan's dates
if len(unique_amounts) > 1 and 'concurrent' in dir() and concurrent and l
    largest_amt = max(unique_amounts)
    freq_txns = [t for t in txns if t['amount'] == largest_amt]
else:
    freq_txns = txns

per_loan_count = len(freq_txns)
if per_loan_count == 1:
    freq = 'monthly'
elif per_loan_count >= 2:
    # Calculate average gap in days (approximate)
    gaps = []
    for i in range(1, len(freq_txns)):
        d1 = date_sort_key(freq_txns[i-1]['date'])
        d2 = date_sort_key(freq_txns[i]['date'])
        m1, day1 = d1 // 100, d1 % 100
        m2, day2 = d2 // 100, d2 % 100
```

```
                approx_days = (m2 - m1) * 30 + (day2 - day1)
                if approx_days > 0:
                    gaps.append(approx_days)
            if gaps:
                avg_gap = sum(gaps) / len(gaps)
                if avg_gap <= 2:
                    freq = 'daily'
                elif avg_gap <= 9:
                    freq = 'weekly'
                elif avg_gap <= 18:
                    freq = 'bi-weekly'
                else:
                    freq = 'monthly'
            else:
                freq = 'daily' if count >= 18 else 'weekly' if count >= 4 else 'b

        positions.append({
            'lender': display_name,
            'amount': most_recent_amt,
            'frequency': freq,
            'occurrence_count': count,
            'confidence': 1.0,
            'notes': notes,
            'monthly_amount': calc_monthly(most_recent_amt, freq)
        })

    # Sort by monthly impact descending
    positions.sort(key=lambda x: x['monthly_amount'], reverse=True)
    return positions

# ============================================================
# VALIDATION LAYER 2: Balance Reconciliation
# ============================================================
def reconcile_balances(raw_text, parsed_months):
    """
    For each month, check: beg_balance + deposits - withdrawals ≈ end_balance
    Returns list of reconciliation results.
    """
    results = []
    # Try to find beginning/ending balances from statement header
    beg_pattern = re.compile(r'[Bb]eginning [Bb]alance\s+\$?([\d,]+\.\d{2})')
    end_pattern = re.compile(r'[Ee]nding [Bb]alance\s+[\d]+\s+\$?([\-\d,]+\.\d{2}
    dep_pattern = re.compile(r'Deposits(?:/Credits)?\s+[\d]+\s+([\d,]+\.\d{2})')
    with_pattern = re.compile(r'(?:Withdrawals?/Debits?|Total Withdrawals?)\s*[-]

    beg_matches = beg_pattern.findall(raw_text)
    end_matches = end_pattern.findall(raw_text)
```

```
    dep_matches = dep_pattern.findall(raw_text)
    with_matches = with_pattern.findall(raw_text)

    for i, month in enumerate(parsed_months):
        if i >= len(beg_matches) or i >= len(end_matches):
            results.append({
                'month': month.get('month_label','?'),
                'status': 'SKIP',
                'note': 'Balance data not found in text'
            })
            continue
        try:
            beg = float(beg_matches[i].replace(',',''))
            end = float(end_matches[i].replace(',','').replace('-',''))
            if end_matches[i].startswith('-'):
                end = -end
            deps = float(dep_matches[i].replace(',','')) if i < len(dep_matches)
            withs = float(with_matches[i].replace(',','')) if i < len(with_matche
            expected = beg + deps - withs
            diff = abs(expected - end)
            if diff < 1.00:
                results.append({'month': month.get('month_label','?'), 'status':
                                'note': 'Reconciles within $1.00'})
            else:
                results.append({'month': month.get('month_label','?'), 'status':
                                'note': 'Discrepancy of ${:,.2f} — review totals
                                'discrepancy': diff})
        except:
            results.append({'month': month.get('month_label','?'), 'status': 'SKI
                            'note': 'Could not parse balance figures'})
    return results

# ============================================================
# VALIDATION LAYER 3: Multi-month Continuity Check
# ============================================================
def check_continuity(raw_text, parsed_months):
    """
    Verify ending balance of month N = beginning balance of month N+1.
    """
    flags = []
    beg_pattern = re.compile(r'[Bb]eginning [Bb]alance\s+\$?([\-\d,]+\.\d{2})')
    end_pattern = re.compile(r'[Ee]nding [Bb]alance\s+\S+\s+\$?([\-\d,]+\.\d{2})'
    begs = beg_pattern.findall(raw_text)
    ends = end_pattern.findall(raw_text)

    def parse_bal(s):
        s = s.replace(',','')
```

```
        return float('-'+s.lstrip('-')) if s.startswith('-') else float(s)

    for i in range(len(ends)-1):
        if i+1 >= len(begs):
            break
        try:
            end_i = parse_bal(ends[i])
            beg_next = parse_bal(begs[i+1])
            diff = abs(end_i - beg_next)
            if diff > 1.00:
                m1 = parsed_months[i].get('month_label','?') if i < len(parsed_mo
                m2 = parsed_months[i+1].get('month_label','?') if i+1 < len(parse
                flags.append({
                    'months': '{} -> {}'.format(m1, m2),
                    'end_bal': end_i,
                    'next_beg_bal': beg_next,
                    'diff': diff,
                    'note': 'Ending balance ${:,.2f} does not match next month op
                })
        except:
            pass
    return flags

# ============================================================
# VALIDATION LAYER 4: Second-pass Claude Verification
# ============================================================
def verify_with_claude(raw_text, parsed_data):
    """
    Independent second Claude review — checks for contradictions and missed items
    Returns a list of review flags.
    """
    client = anthropic.Anthropic()
    positions_summary = "\n".join([
        "  - {}: ${:,.2f} {} (monthly=${:,.2f})".format(
            p.get('lender'), p.get('amount',0), p.get('frequency',''),
            calc_monthly(p.get('amount',0), p.get('frequency','weekly'))
        ) for p in parsed_data.get('current_positions', [])
    ])
    months_summary = "\n".join([
        "  - {}: total_deposits=${:,.2f} true_deposits=${:,.2f} adb=${:,.2f} nsf=
            m.get('month_label'), m.get('total_deposits',0),
            m.get('true_deposits',0), m.get('adb',0), m.get('nsf_count',0)
        ) for m in parsed_data.get('months', [])
    ])

    prompt = (
        "You are a second independent MCA underwriter reviewing extracted bank st
```

```
        "Your job is ONLY to find errors, contradictions, or missed items in the
        "Return ONLY valid JSON array, no markdown.\n\n"
        "SOURCE STATEMENT TEXT (first 20000 chars):\n"
        + raw_text[:20000] +
        "\n\nEXTRACTED DATA TO VERIFY:\n"
        "Current Positions:\n" + positions_summary +
        "\nMonthly Data:\n" + months_summary +
        "\n\nCheck for:\n"
        "1. Any ACH debits in the text NOT in the positions list — even if only 1
        "2. Total deposits in text that don't match extracted values (>$500 diffe
        "3. NSF/returned items in text not counted\n"
        "4. MCA funding wires included in true_deposits that should be excluded\n
        "5. Wrong frequency — verify by counting actual gaps between payment date
        "6. ADB significantly different from what Interest Summary shows\n"
        "7. Payments that STOPPED mid-statement with no note (lender in early mon
        "8. Payment amounts that CHANGED with no note (same lender, different amo
        "9. Same company listed as two separate positions when they should be com
        "10. Single-occurrence debits that were missed because they appeared only
        "Return JSON array of issues found. Empty array [] if no issues.\n"
        "Each issue: {\"type\": \"MISSING_POSITION|WRONG_AMOUNT|WRONG_FREQUENCY|W
        " \"description\": \"clear explanation with dates and amounts\", \"confid
        "Only flag real issues you can verify in the source text."
    )

    msg = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=2000,
        messages=[{"role":"user","content":prompt}]
    )
    raw = msg.content[0].text.strip()
    raw = re.sub(r'^```json\s*','',raw)
    raw = re.sub(r'^```\s*','',raw)
    raw = re.sub(r'\s*```$','',raw)
    try:
        issues = json.loads(raw)
        if not isinstance(issues, list):
            issues = []
    except:
        issues = []
    return issues

# ============================================================
# MAIN PARSE FUNCTION
# ============================================================
def parse_with_claude(raw_text, company_name=""):
    client = anthropic.Anthropic()
    cn = company_name if company_name else 'auto-detect'
```

```
prompt = (
    "You are an expert MCA (Merchant Cash Advance) underwriter analyzing bank
    "You are the LENDER deciding whether to advance money to this business.\n
    "Return ONLY valid JSON, no markdown, no explanation.\n\n"
    "BANK STATEMENT TEXT:\n"
    + raw_text[:80000] +
    "\n\n"
    "=== CRITICAL EXTRACTION RULES ===\n\n"
    "RULE 1 - CURRENT POSITIONS — READ EVERY TRANSACTION INDIVIDUALLY:\n"
    "You MUST read every single ACH debit transaction line by line. Do NOT as
    "Do NOT infer frequency from just a few instances — count every actual oc
    "For each unique lender/payee that appears as a debit:\n"
    "  a) List every date and amount they debited across all months\n"
    "  b) amount = the most recent per-payment amount (use the last occurrenc
    "  c) frequency = determined ONLY by counting actual gaps between payment
    "     - Debits on consecutive business days = 'daily'\n"
    "     - Debits ~7 days apart = 'weekly'\n"
    "     - Debits ~14 days apart = 'bi-weekly'\n"
    "     - Debits ~30 days apart = 'monthly'\n"
    "  d) If payments STOPPED mid-statement, note it: 'stopped after MM/DD'\n
    "  e) If payment AMOUNT CHANGED, note both amounts: 'was $X, changed to $
    "  f) If only 1-2 payments exist across all months, still include it — do
    "  g) If the final months show ZERO debits from a lender that appeared ea
    "     note it as 'possibly completed/paid off — last payment MM/DD'\n\n"
    "DEDUPLICATION RULE — CRITICAL:\n"
    "If the same lender appears with the same payment amount debiting on a re
    "it is ONE position. Do NOT list it twice. The reference numbers and date
    "the transaction description change every payment but it is still the sam
    "Example: 'Legacy Fund Cs1507 Feb04' and 'Legacy Fund Cs1507 Feb05' at $1
    "= ONE position: Legacy Fund, $1,206.02 daily. Not two positions.\n\n"
    "SAME COMPANY MULTIPLE LOANS — COMBINE THEM:\n"
    "  - If the SAME company appears with MULTIPLE different debit IDs or amo
    "    (e.g. 'OnDeck Capital xxxxx1234' and 'OnDeck Capital xxxxx5678'),\n"
    "    these are separate loans from the same lender.\n"
    "  - Combine them into ONE position entry for that lender.\n"
    "  - amount = combined total per payment cycle (sum both loan amounts)\n"
    "  - notes = 'Two loans: $X + $Y = $Z per payment'\n"
    "  - Do NOT create two separate rows for the same company.\n\n"
    "RULE 2 - TRUE DEPOSITS (only real business revenue):\n"
    "INCLUDE: POS/credit card processor deposits (Stripe, Lightspeed, Square,
    "eDeposit IN Branch, Mobile Deposits, ACH credits from real customers/ven
    "EXCLUDE: MCA funding credits (DC suffix on MCA names, wire credits from
    "Fiji SPV LLC wire, Online transfers FROM personal accounts, Book transfe
    "Returned item credits\n\n"
    "RULE 3 - LEVERAGE PERCENTAGE:\n"
    "  leverage_pct = (sum of all ACTIVE monthly MCA payment amounts / true m
    "  Use the most recent FULL month. Only count lenders still actively debi
```

```
    "RULE 4 - NSF / RETURNED ITEMS:\n"
    "  Count every entry in 'Items returned unpaid' section and every 'Overdr
    "  nsf_count = items in 'Items returned unpaid'\n"
    "  od_count = number of 'Overdraft Fee' lines\n\n"
    "RULE 5 - MONTHS:\n"
    "  Extract EVERY statement period. Each has a 'Statement period activity
    "  total_deposits = Deposits/Credits total from the summary box\n"
    "  adb = Average collected balance from Interest summary\n\n"
    "Return this exact JSON structure:\n"
    '{"company_name":"string","account_number_last4":"string","num_bank_accou
    '"offer_decline":"DECLINE","holdback_pct":0.0,"leverage_pct":0.0,'
    '"sos_info":"","court_search_notes":"",'
    '"account_notes":[],'
    '"current_positions":['
    '{"lender":"name","amount":0.0,"frequency":"daily/weekly/bi-weekly/monthl
    '],'
    '"months":['
    '{"month_label":"Mon-YY","period":"MM/DD to MM/DD","is_mtd":false,'
    '"total_deposits":0.0,"total_deposits_confidence":0.9,'
    '"true_deposits":0.0,"true_deposits_confidence":0.9,"true_deposit_exclusi
    '"neg_days":0,"nsf_count":0,"od_count":0,"num_transactions":0,'
    '"adb":0.0,"adb_confidence":0.9,"days_below_1000":0,'
    '"funding_events":[{"funder":"name","amount":0.0,"date":"MM/DD"}],'
    '"notes":""}'
    ']}\n\n'
    "Company name if provided: " + cn
)
msg = client.messages.create(
    model="claude-opus-4-5",
    max_tokens=8000,
    messages=[{"role":"user","content":prompt}]
)
raw = msg.content[0].text.strip()
raw = re.sub(r'^```json\s*','',raw)
raw = re.sub(r'^```\s*','',raw)
raw = re.sub(r'\s*```$','',raw)
data = json.loads(raw)

# Sanitize AI output immediately
def clean_json(obj):
    if isinstance(obj, dict):
        return {k: clean_json(v) for k, v in obj.items()}
    elif isinstance(obj, list):
        return [clean_json(i) for i in obj]
    elif isinstance(obj, str):
        result = []
        for ch in obj:
```

```
            cp = ord(ch)
            if cp < 0x20 and cp not in (0x09, 0x0a, 0x0d): continue
            if 0xD800 <= cp <= 0xDFFF: continue
            if 0xFFFE <= cp <= 0xFFFF: continue
            result.append(ch)
        s = ''.join(result)
        s = s.replace('\u2013','-').replace('\u2014','-')
        s = s.replace('\u2018',"'").replace('\u2019',"'")
        s = s.replace('\u201c','"').replace('\u201d','"')
        s = s.replace('\u2022','*').replace('\u00a0',' ')
        return s
    return obj
data = clean_json(data)

# DEDUPLICATION — remove duplicate positions from AI output
# Same lender + same amount + same frequency = one position, not two
def dedup_positions(positions):
    seen = {}
    deduped = []
    for pos in positions:
        lender = pos.get('lender', '').strip().lower()
        amount = round(pos.get('amount', 0), 2)
        freq = pos.get('frequency', '').lower()

        # Normalize lender name — strip trailing reference numbers/codes
        # so "Legacy Fund Cs1234" and "Legacy Fund Cs5678" both become "legac
        lender_normalized = re.sub(r'\s+[a-z0-9#]{4,}\s*$', '', lender, flags
        lender_normalized = re.sub(r'\s+\d{6}\s*$', '', lender_normalized).st
        # Use first 3 words as key
        key_words = lender_normalized.split()[:3]
        key = (' '.join(key_words), amount, freq)

        if key not in seen:
            seen[key] = pos
            deduped.append(pos)
        else:
            # Already have this lender+amount+freq — merge notes if different
            existing = seen[key]
            existing_notes = existing.get('notes', '')
            new_notes = pos.get('notes', '')
            if new_notes and new_notes not in existing_notes:
                existing['notes'] = (existing_notes + '; ' + new_notes).strip
    return deduped

data['current_positions'] = dedup_positions(data.get('current_positions', []))

# Calculate monthly totals
```

```
    total = 0
    for pos in data.get("current_positions", []):
        monthly = calc_monthly(pos.get("amount", 0), pos.get("frequency", "weekly
        pos["monthly_amount"] = monthly
        total += monthly
    data["total_current_positions"] = total

    # Calculate leverage % if not set
    if not data.get("leverage_pct") and data.get("months"):
        full_months = [m for m in data["months"] if not m.get("is_mtd")]
        if full_months:
            true_dep = full_months[0].get("true_deposits", 0)
            if true_dep > 0:
                data["leverage_pct"] = round((total / true_dep) * 100, 2)

    return data

def merge_data(existing, new_data):
    existing_labels = {m['month_label'] for m in existing.get('months', [])}
    for m in new_data.get('months', []):
        if m['month_label'] not in existing_labels:
            existing['months'].append(m)

    def month_sort_key(m):
        label = m.get('month_label', '')
        month_map = {'Jan':1,'Feb':2,'Mar':3,'Apr':4,'May':5,'Jun':6,
                     'Jul':7,'Aug':8,'Sep':9,'Oct':10,'Nov':11,'Dec':12}
        try:
            parts = label.split('-')
            return int(parts[1]) * 100 + month_map.get(parts[0], 0)
        except:
            return 0
    existing['months'].sort(key=month_sort_key, reverse=True)

    existing_lenders = {p['lender'] for p in existing.get('current_positions', []
    for p in new_data.get('current_positions', []):
        if p['lender'] not in existing_lenders:
            existing['current_positions'].append(p)
    total = sum(calc_monthly(p.get('amount',0), p.get('frequency','weekly'))
                for p in existing.get('current_positions', []))
    existing['total_current_positions'] = total
    return existing

# ============================================================
# EXCEL BUILDER
# ============================================================
def build_excel(data):
```

```
wb = openpyxl.Workbook()
ws = wb.active
ws.title = "Analysis"

GOLD_PALE="FFFFF8E1"; LIGHT_YELLOW="FFFFFF99"; DARK_GOLD="FF8B6914"
GREEN_BG="FFC6EFCE"; GREEN_FG="FF006100"; RED_BG="FFFFC7CE"; RED_FG="FF9C0006
BLUE="FF0070C0"; PURPLE_BG="FFEAD5F5"; GRAY="FFF2F2F2"
NEG_BG="FFFFC7CE"; OK_BG="FFC6EFCE"; MONTH_BG="FFFFD966"
YELLOW_FLAG="FFFFFF99"; ORANGE_FLAG="FFFFD28C"

thin = Side(style='thin')
def border_all():
    return Border(left=thin,right=thin,top=thin,bottom=thin)

def safe_str(val):
    if val is None: return ""
    s = str(val)
    s = s.replace('\u2013','-').replace('\u2014','-')
    s = s.replace('\u2018',"'").replace('\u2019',"'")
    s = s.replace('\u201c','"').replace('\u201d','"')
    s = s.replace('\u2022','*').replace('\u00a0',' ')
    result = []
    for ch in s:
        cp = ord(ch)
        if cp < 0x20 and cp not in (0x09, 0x0a, 0x0d): continue
        if 0xD800 <= cp <= 0xDFFF: continue
        if 0xFFFE <= cp <= 0xFFFF: continue
        result.append(ch)
    return ''.join(result)

def w(row,col,value="",bold=False,sz=10,color=None,bg=None,align="left",
      bdr=False,italic=False,ul=False,wrap=False,flag=False):
    if isinstance(value, str): value = safe_str(value)
    c = ws.cell(row=row,column=col,value=value)
    kw={"bold":bold,"size":sz,"italic":italic}
    if ul: kw["underline"]="single"
    if color: kw["color"]=color
    c.font=Font(**kw)
    actual_bg = YELLOW_FLAG if flag else bg
    if actual_bg: c.fill=PatternFill("solid",start_color=actual_bg)
    c.alignment=Alignment(horizontal=align,vertical="center",wrap_text=wrap)
    if bdr: c.border=border_all()
    return c

def merge(r1,c1,r2,c2):
    try:
        ws.merge_cells(start_row=r1,start_column=c1,end_row=r2,end_column=c2)
```

```
    except:
        pass

for col,wd in {1:3,2:34,3:20,4:16,5:16,6:14,7:18,8:40}.items():
    ws.column_dimensions[get_column_letter(col)].width=wd

row=1
ws.row_dimensions[row].height=6; row+=1

# Header row
ws.row_dimensions[row].height=26
w(row,3,"Amounts ($) / No.",bold=True,sz=9,align="center",bg=GRAY)
w(row,4,"frequency",sz=9,align="center",bg=GRAY,italic=True)
merge(row,5,row,5)
c=ws.cell(row=row,column=5,value="APPROVED")
c.font=Font(bold=True,size=10,color=GREEN_FG)
c.fill=PatternFill("solid",start_color=GREEN_BG)
c.alignment=Alignment(horizontal="center",vertical="center")
c.border=border_all()
ac = ws.cell(row=row,column=6,value="")
ac.font=Font(bold=True,size=12,color=GREEN_FG)
ac.fill=PatternFill("solid",start_color=GREEN_BG)
ac.alignment=Alignment(horizontal="center",vertical="center")
ac.border=border_all()
merge(row,7,row+1,8)
c=ws.cell(row=row,column=7,value="Update Sheet\nTab Color")
c.font=Font(bold=True,size=11)
c.fill=PatternFill("solid",start_color=PURPLE_BG)
c.alignment=Alignment(horizontal="center",vertical="center",wrap_text=True)
row+=1

ws.row_dimensions[row].height=22
c=ws.cell(row=row,column=5,value="DECLINED")
c.font=Font(bold=True,size=10,color=RED_FG)
c.fill=PatternFill("solid",start_color=RED_BG)
c.alignment=Alignment(horizontal="center",vertical="center")
c.border=border_all()
dc = ws.cell(row=row,column=6,value="")
dc.font=Font(bold=True,size=12,color=RED_FG)
dc.fill=PatternFill("solid",start_color=RED_BG)
dc.alignment=Alignment(horizontal="center",vertical="center")
dc.border=border_all()
row+=1

ws.row_dimensions[row].height=6; row+=1
ws.row_dimensions[row].height=18
w(row,5,"",sz=16,align="center"); row+=1
```

```
# Company name
ws.row_dimensions[row].height=24
merge(row,2,row,6)
c=ws.cell(row=row,column=2,value=safe_str(data.get("company_name","COMPANY NA
c.font=Font(bold=True,size=13,color=DARK_GOLD)
c.fill=PatternFill("solid",start_color=LIGHT_YELLOW)
c.alignment=Alignment(horizontal="left",vertical="center")
row+=1

# Offer/Decline
ws.row_dimensions[row].height=20
w(row,2,"OFFER / DECLINE",bold=True,ul=True,sz=10)
w(row,3,"$0.00",sz=10,color=BLUE)
w(row,4,"daily",sz=9,italic=True)
w(row,5,"No. of Bank Accts",bold=True,sz=9)
w(row,6,data.get("num_bank_accounts",1),bold=True,sz=11,align="center")
row+=1

for note in data.get("account_notes",[])[:4]:
    ws.row_dimensions[row].height=15
    clr="FFCC0000" if any(x in note.lower() for x in ["1,000","negative","nsf
    w(row,2,"*"+note,sz=9,italic=True,color=clr); row+=1
for _ in range(max(0,3-len(data.get("account_notes",[])))):
    ws.row_dimensions[row].height=14; row+=1

# Holdback, leverage
hb=data.get("holdback_pct",0)
lv=data.get("leverage_pct",0)
ws.row_dimensions[row].height=18
w(row,3,"Holdback %",bold=True,sz=10,align="right")
w(row,4,"{:.2f}%".format(hb) if hb else "",sz=10,align="center"); row+=1

ws.row_dimensions[row].height=18
w(row,2,"SOS",bold=True,ul=True,sz=10)
w(row,3,"New Holdback %",bold=True,sz=10,align="right")
w(row,4,"{:.2f}%".format(hb) if hb else "",sz=10,align="center"); row+=1

lv_color = "FFCC0000" if lv > 50 else "FF006100"
lv_bg = "FFFFC7CE" if lv > 80 else (YELLOW_FLAG if lv > 50 else None)
ws.row_dimensions[row].height=18
w(row,2,"Leverage %",bold=True,sz=10,color=lv_color)
w(row,3,"{:.2f}%".format(lv) if lv else "0.00%",bold=True,sz=11,align="center
  color=lv_color,bg=lv_bg); row+=1

ws.row_dimensions[row].height=16
sos=data.get("sos_info","")
```

```
w(row,2,sos if sos else "Active MM/DD/YYYY",sz=9,italic=True); row+=2

ws.row_dimensions[row].height=18
w(row,2,"Court Search",bold=True,ul=True,sz=10); row+=1
ws.row_dimensions[row].height=16
court=data.get("court_search_notes","")
w(row,2,court if court else "*No court records found",sz=9,italic=True,wrap=T

# Account number
ws.row_dimensions[row].height=20
acct=data.get("account_number_last4","")
merge(row,2,row,4)
c=ws.cell(row=row,column=2,value=safe_str(acct) if acct else "ACCOUNT DETAILS
c.font=Font(bold=True,size=12,color=DARK_GOLD)
c.fill=PatternFill("solid",start_color=GOLD_PALE)
c.alignment=Alignment(horizontal="left",vertical="center")
row+=2

# Current Positions
ws.row_dimensions[row].height=18
w(row,2,"Current Positions:",bold=True,ul=True,sz=10)
total=data.get("total_current_positions",0)
w(row,3,"${:,.2f}".format(total) if total else "$0.00",bold=True,sz=10,color=
w(row,5,"(monthly total)",sz=8,italic=True,color="FF808080"); row+=1

for pos in data.get("current_positions",[]):
    ws.row_dimensions[row].height=15
    lender=safe_str(pos.get("lender",""))
    amt=pos.get("amount",0)
    freq=safe_str(pos.get("frequency","weekly"))
    notes=safe_str(pos.get("notes",""))
    monthly=pos.get("monthly_amount", calc_monthly(amt, freq))
    conf=pos.get("confidence",1.0)
    is_flagged = conf < 0.85
    w(row,2,lender,sz=9,color=BLUE,flag=is_flagged)
    w(row,3,"${:,.2f}".format(amt) if amt else "",sz=9,align="right",flag=is_
    if freq: w(row,4,"*"+freq,sz=9,italic=True,flag=is_flagged)
    w(row,5,"= ${:,.2f}/mo".format(monthly),sz=8,italic=True,color="FF808080"
    if is_flagged: w(row,7,"⚑ conf={:.0f}%".format(conf*100),sz=8,italic=True
    if notes: w(row,6,"*"+notes,sz=9,italic=True,color="FFCC0000")
    row+=1

ws.row_dimensions[row].height=16
w(row,2,"Other Loans / Positions:",bold=True,sz=10); row+=2

# Monthly sections
def month_sort_key(m):
```

```
    label = m.get('month_label', '')
    month_map = {'Jan':1,'Feb':2,'Mar':3,'Apr':4,'May':5,'Jun':6,
                 'Jul':7,'Aug':8,'Sep':9,'Oct':10,'Nov':11,'Dec':12}
    try:
        parts = label.split('-')
        return int(parts[1]) * 100 + month_map.get(parts[0], 0)
    except:
        return 0
months_sorted = sorted(data.get("months",[]), key=month_sort_key, reverse=Tru

for m in months_sorted:
    label=safe_str(m.get("month_label",""))
    period=safe_str(m.get("period",""))
    is_mtd=m.get("is_mtd",False)

    ws.row_dimensions[row].height=22
    merge(row,2,row,7)
    hdr="{} (MTD) From {}".format(label,period) if is_mtd and period else lab
    c=ws.cell(row=row,column=2,value=hdr)
    c.font=Font(bold=True,size=11)
    c.fill=PatternFill("solid",start_color=MONTH_BG)
    c.alignment=Alignment(horizontal="left",vertical="center")
    row+=1

    # Total deposits
    td=m.get("total_deposits",0)
    td_conf=m.get("total_deposits_confidence",1.0)
    td_flag = td_conf < 0.85
    ws.row_dimensions[row].height=16
    w(row,2,"Total deposits:",sz=10)
    w(row,3,"${:,.2f}".format(td),sz=10,color=BLUE,flag=td_flag)
    if is_mtd: w(row,4,"*calculated *",sz=9,italic=True,color="FF808080")
    if td_flag: w(row,7,"⚑ conf={:.0f}%".format(td_conf*100),sz=8,italic=True
    row+=1

    # True deposits
    trd=m.get("true_deposits",0)
    trd_conf=m.get("true_deposits_confidence",1.0)
    trd_flag = trd_conf < 0.85
    lbl="True deposits (MTD):" if is_mtd else "True deposits:"
    ws.row_dimensions[row].height=16
    w(row,2,lbl,sz=10)
    w(row,3,"${:,.2f}".format(trd),sz=10,color=BLUE,flag=trd_flag)
    ntx=m.get("num_transactions",0)
    if is_mtd and ntx: w(row,4,str(ntx),sz=10,align="center",color=BLUE)
    excl=safe_str(m.get("true_deposit_exclusions",""))
    if excl: w(row,5,"*excl. "+excl,sz=8,italic=True,color="FF808080",wrap=Tr
```

```
if trd_flag: w(row,7,"⚑ conf={:.0f}%".format(trd_conf*100),sz=8,italic=Tr
row+=1

# Neg/NSF/OD bar
neg=m.get("neg_days",0); nsf=m.get("nsf_count",0); od=m.get("od_count",0)
bar_label="Neg days # {} / NSF # {} / OD # {}".format(neg,nsf,od)
bar_bg=NEG_BG if (neg>0 or nsf>0 or od>0) else OK_BG
bar_fg=RED_FG if (neg>0 or nsf>0 or od>0) else GREEN_FG
merge(row,2,row,5)
c=ws.cell(row=row,column=2,value=bar_label)
c.font=Font(bold=True,size=9,color=bar_fg)
c.fill=PatternFill("solid",start_color=bar_bg)
c.alignment=Alignment(horizontal="left",vertical="center")
row+=1

# ADB
adb=m.get("adb",0)
adb_conf=m.get("adb_confidence",1.0)
adb_flag = adb_conf < 0.85
ws.row_dimensions[row].height=16
w(row,2,"ADB (average daily balance)",sz=10)
w(row,3,"${:,.2f}".format(adb),sz=10,color=BLUE,flag=adb_flag)
w(row,4,"*given" if adb else "*calculated",sz=9,italic=True,color="FF8080
if adb_flag: w(row,7,"⚑ conf={:.0f}%".format(adb_conf*100),sz=8,italic=Tr
row+=1

# Days below 1000
dl=m.get("days_below_1000",0)
ws.row_dimensions[row].height=16
w(row,2,"Days below $1,000:",sz=10)
w(row,3,str(dl),sz=10,align="center"); row+=1

# Funding events
for fe in m.get("funding_events",[]):
    w(row,2,"*Funded by "+safe_str(fe.get("funder","")),sz=9,italic=True,
    amt_fe=fe.get("amount",0)
    w(row,3,"with an amount of ${:,.2f}".format(amt_fe) if amt_fe else ""
    dt=safe_str(fe.get("date",""))
    if dt: w(row,4,"on "+dt,sz=9,italic=True)
    row+=1

mnotes=safe_str(m.get("notes",""))
if mnotes:
    w(row,2,"*"+mnotes,sz=9,italic=True,color="FF808080",wrap=True); row+

row+=2
```

```
# ============================================================
# REVIEW FLAGS SECTION
# ============================================================
review_flags = data.get("review_flags", [])
recon_results = data.get("reconciliation_results", [])
continuity_flags = data.get("continuity_flags", [])
verify_issues = data.get("verify_issues", [])

all_issues = []
for r in recon_results:
    if r.get('status') == 'FLAG':
        all_issues.append(("RECON", r.get('month','?'), r.get('note','')))
for c_flag in continuity_flags:
    all_issues.append(("CONTINUITY", c_flag.get('months','?'), c_flag.get('no
for v in verify_issues:
    all_issues.append((v.get('type','VERIFY'), 'ALL', v.get('description','')
for rf in review_flags:
    all_issues.append((rf.get('type','FLAG'), rf.get('month','?'), rf.get('re

row += 1
ws.row_dimensions[row].height=20
merge(row,2,row,8)
c=ws.cell(row=row,column=2,value="REVIEW FLAGS")
c.font=Font(bold=True,size=12)
c.fill=PatternFill("solid",start_color="FF2D2D2D")
c.font=Font(bold=True,size=12,color="FFFFFFFF")
c.alignment=Alignment(horizontal="left",vertical="center")
row+=1

if not all_issues:
    ws.row_dimensions[row].height=18
    merge(row,2,row,8)
    c=ws.cell(row=row,column=2,value="✓ No review flags — all checks passed")
    c.font=Font(bold=True,size=10,color=GREEN_FG)
    c.fill=PatternFill("solid",start_color=OK_BG)
    c.alignment=Alignment(horizontal="left",vertical="center")
    row+=1
else:
    # Header
    ws.row_dimensions[row].height=16
    w(row,2,"Type",bold=True,sz=9,bg=GRAY)
    w(row,3,"Month",bold=True,sz=9,bg=GRAY)
    merge(row,4,row,8)
    c=ws.cell(row=row,column=4,value="Issue / Action Required")
    c.font=Font(bold=True,size=9)
    c.fill=PatternFill("solid",start_color=GRAY)
    c.alignment=Alignment(horizontal="left",vertical="center")
```

```
        row+=1

        for issue_type, month, desc in all_issues:
            ws.row_dimensions[row].height=15
            bg = YELLOW_FLAG if issue_type not in ("RECON","CONTINUITY") else ORA
            w(row,2,issue_type,sz=9,bg=bg)
            w(row,3,str(month),sz=9,bg=bg)
            merge(row,4,row,8)
            c=ws.cell(row=row,column=4,value=safe_str(desc))
            c.font=Font(size=9,italic=True)
            c.fill=PatternFill("solid",start_color=bg)
            c.alignment=Alignment(horizontal="left",vertical="center",wrap_text=T
            row+=1

    ws.freeze_panes="B7"
    out=io.BytesIO(); wb.save(out); out.seek(0)
    return out

# ============================================================
# ROUTES
# ============================================================
@app.route('/login', methods=['GET','POST'])
def login():
    error = None
    if request.method == 'POST':
        username = request.form.get('username','').strip()
        password = request.form.get('password','').strip()
        if USERS.get(username) == password:
            session['logged_in'] = True
            session['username'] = username
            return redirect(url_for('index'))
        error = 'Invalid username or password'
    return render_template('login.html', error=error)

@app.route('/logout')
def logout():
    session.clear()
    return redirect(url_for('login'))

@app.route('/')
@login_required
def index():
    history = load_history()
    return render_template('index.html', history=history)

@app.route('/analyze', methods=['POST'])
@login_required
```

```
def analyze():
    if 'files' not in request.files: return jsonify({"error":"No files uploaded"}
    files=request.files.getlist('files')
    company_name=request.form.get('company_name','')
    entry_id=request.form.get('entry_id','')
    if not files or all(f.filename=='' for f in files): return jsonify({"error":"

    combined_text=""
    for file in files:
        if file and allowed_file(file.filename):
            fname=secure_filename(file.filename)
            fpath=os.path.join(app.config['UPLOAD_FOLDER'],fname)
            file.save(fpath)
            try:
                combined_text+="\n\n=== FILE: {} ===\n".format(fname)+extract_tex
            except Exception as e:
                return jsonify({"error":"Failed to read {}: {}".format(fname,str(
            finally:
                if os.path.exists(fpath): os.remove(fpath)
        else: return jsonify({"error":"Unsupported file: {}".format(file.filename

    if not combined_text.strip(): return jsonify({"error":"No text extracted"}),4

    try: new_data=parse_with_claude(combined_text,company_name)
    except Exception as e: return jsonify({"error":"AI parsing failed: {}".format

    new_data = sanitize_data(new_data)

    # Run deterministic rules engine and merge any new positions found
    try:
        rules_positions = run_rules_engine(combined_text)
        existing_lenders = set()
        for p in new_data.get('current_positions', []):
            lender = p['lender'].lower()
            lender_key = ' '.join(re.sub(r'\s+[a-z0-9#]{4,}\s*$', '', lender).spl
            existing_lenders.add(lender_key)
        for rp in rules_positions:
            lender = rp['lender'].lower()
            lender_key = ' '.join(re.sub(r'\s+[a-z0-9#]{4,}\s*$', '', lender).spl
            if lender_key not in existing_lenders:
                new_data.setdefault('current_positions',[]).append({
                    'lender': rp['lender'],
                    'amount': rp['amount'],
                    'frequency': rp['frequency'],
                    'confidence': 1.0,
                    'notes': rp['notes'],
                    'monthly_amount': calc_monthly(rp['amount'], rp['frequency'])
```

```
                })
    except: pass

    # Balance reconciliation
    try:
        new_data['reconciliation_results'] = reconcile_balances(combined_text, ne
    except: pass

    # Multi-month continuity check
    try:
        new_data['continuity_flags'] = check_continuity(combined_text, new_data.g
    except: pass

    # Second-pass Claude verification
    try:
        new_data['verify_issues'] = verify_with_claude(combined_text, new_data)
    except: pass

    if entry_id:
        existing = load_entry(entry_id)
        if existing:
            new_data = merge_data(existing['data'], new_data)

    try:
        excel=build_excel(new_data)
        excel_bytes=excel.read()
    except Exception as e:
        return jsonify({"error":"Excel generation failed: {}".format(str(e))}),50

    cn=new_data.get("company_name","Unknown")
    save_history(cn, new_data, excel_bytes)
    safe=re.sub(r'[^\w\s-]','',cn).strip().replace(' ','_')
    return send_file(io.BytesIO(excel_bytes),as_attachment=True,
                     download_name=safe+"_analysis.xlsx",
                     mimetype='application/vnd.openxmlformats-officedocument.spre

@app.route('/history/<entry_id>/download')
@login_required
def download_history(entry_id):
    entry = load_entry(entry_id)
    if not entry: return "Not found", 404
    try:
        clean_data = sanitize_data(entry['data'])
        excel = build_excel(clean_data)
        excel_bytes = excel.read()
    except:
        excel_bytes = entry['excel']
```

```
    safe=re.sub(r'[^\w\s-]','',entry['company_name']).strip().replace(' ','_')
    return send_file(io.BytesIO(excel_bytes),as_attachment=True,
                     download_name=safe+"_analysis.xlsx",
                     mimetype='application/vnd.openxmlformats-officedocument.spre

@app.route('/history/<entry_id>/delete', methods=['POST'])
@login_required
def delete_history(entry_id):
    path = os.path.join(app.config['HISTORY_FOLDER'], entry_id + '.pkl')
    if os.path.exists(path): os.remove(path)
    return redirect(url_for('index'))


# ============================================================
# SUPABASE MEMORY HELPERS
# ============================================================
SUPABASE_URL = os.environ.get('SUPABASE_URL', '')
SUPABASE_KEY = os.environ.get('SUPABASE_KEY', '')

def supabase_request(method, endpoint, data=None):
    """Make a request to Supabase REST API."""
    url = SUPABASE_URL.rstrip('/') + '/rest/v1/' + endpoint
    headers = {
        'apikey': SUPABASE_KEY,
        'Authorization': 'Bearer ' + SUPABASE_KEY,
        'Content-Type': 'application/json',
        'Prefer': 'return=representation'
    }
    body = json.dumps(data).encode('utf-8') if data else None
    req = urllib.request.Request(url, data=body, headers=headers, method=method)
    try:
        with urllib.request.urlopen(req, timeout=10) as resp:
            return json.loads(resp.read().decode('utf-8'))
    except urllib.error.HTTPError as e:
        err = e.read().decode('utf-8')
        print("Supabase error {}: {}".format(e.code, err))
        return None
    except Exception as ex:
        print("Supabase request failed:", ex)
        return None

def company_key_from_name(name):
    """Normalize company name to a consistent key for matching."""
    key = name.lower().strip()
    key = re.sub(r'[^a-z0-9 ]', '', key)
    key = re.sub(r'\s+', ' ', key).strip()
    return key
```

```
def supabase_get_analysis(company_name):
    """Look up existing analysis by company name."""
    key = company_key_from_name(company_name)
    result = supabase_request('GET',
        'mca_analyses?company_key=eq.{}&order=updated_at.desc&limit=1'.format(
            urllib.parse.quote(key)))
    if result and len(result) > 0:
        row = result[0]
        return row['id'], row['entry_data']
    return None, None

def supabase_save_analysis(company_name, data, excel_bytes):
    """Save or update analysis in Supabase."""
    key = company_key_from_name(company_name)
    row_id, existing = supabase_get_analysis(company_name)

    payload = {
        'company_name': company_name,
        'company_key': key,
        'entry_data': data,
        'updated_at': datetime.now().isoformat()
    }

    if row_id:
        # Update existing
        supabase_request('PATCH',
            'mca_analyses?id=eq.{}'.format(row_id),
            payload)
        return row_id
    else:
        # Insert new
        result = supabase_request('POST', 'mca_analyses', payload)
        if result and len(result) > 0:
            return result[0]['id']
    return None

# ============================================================
# MAILGUN EMAIL HELPER
# ============================================================
def send_email_with_excel(to_addresses, company_name, excel_bytes, month_count, i
    """Send Excel analysis via Mailgun API."""
    mailgun_domain = os.environ.get('MAILGUN_DOMAIN', '')
    mailgun_key = os.environ.get('MAILGUN_API_KEY', '')

    if not mailgun_domain or not mailgun_key:
        print("Mailgun not configured")
```

```
        return False

    subject = "{} - MCA Analysis{}".format(
        company_name,
        " (Updated)" if is_update else ""
    )

    body = """MCA Bank Statement Analysis — {}

{}
Months analyzed: {}
Generated: {}

Please find the Excel analysis attached.

—
AURUM Studio MCA Analyzer
""".format(
        company_name,
        "This analysis has been updated with new statements." if is_update else "
        month_count,
        datetime.now().strftime('%B %d, %Y at %I:%M %p')
    )

    safe_name = re.sub(r'[^\w\s-]', '', company_name).strip().replace(' ', '_')
    filename = safe_name + '_analysis.xlsx'

    # Build multipart form data for Mailgun API
    boundary = '----FormBoundary' + os.urandom(8).hex()

    def add_field(name, value):
        return ('--' + boundary + '\r\n'
                'Content-Disposition: form-data; name="{}"\r\n\r\n'
                '{}\r\n').format(name, value).encode('utf-8')

    def add_file(name, filename, content, content_type):
        return ('--' + boundary + '\r\n'
                'Content-Disposition: form-data; name="{}"; filename="{}"\r\n'
                'Content-Type: {}\r\n\r\n').encode('utf-8') + content + b'\r\n'

    parts = []
    # Add to each recipient
    for addr in to_addresses:
        parts.append(add_field('to', addr))
    parts.append(add_field('from', 'MCA Analyzer <analyze@{}>'.format(mailgun_dom
    parts.append(add_field('subject', subject))
    parts.append(add_field('text', body))
```

```
    parts.append(add_file('attachment', filename, excel_bytes,
        'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet'))
    parts.append(('--' + boundary + '--\r\n').encode('utf-8'))

    body_bytes = b''.join(parts)

    url = 'https://api.mailgun.net/v3/{}/messages'.format(mailgun_domain)
    credentials = base64.b64encode(('api:' + mailgun_key).encode('utf-8')).decode

    req = urllib.request.Request(
        url,
        data=body_bytes,
        headers={
            'Authorization': 'Basic ' + credentials,
            'Content-Type': 'multipart/form-data; boundary=' + boundary,
            'Content-Length': str(len(body_bytes))
        },
        method='POST'
    )

    try:
        with urllib.request.urlopen(req, timeout=30) as resp:
            result = json.loads(resp.read().decode('utf-8'))
            print("Email sent:", result.get('message', 'OK'))
            return True
    except urllib.error.HTTPError as e:
        print("Mailgun send error {}: {}".format(e.code, e.read().decode('utf-8')
        return False
    except Exception as ex:
        print("Email send failed:", ex)
        return False

# ============================================================
# EMAIL INBOUND WEBHOOK ROUTE
# ============================================================
@app.route('/email-inbound', methods=['POST'])
def email_inbound():
    """
    Mailgun inbound parse webhook.
    Subject line = company name.
    Attachment = bank statement PDF.
    Sends Excel reply to REPLY_TO_EMAIL addresses.
    """
    try:
        # Extract company name from subject
        subject = request.form.get('subject', '').strip()
        if not subject:
```

```
    print("Email received with no subject — ignoring")
    return jsonify({"status": "ignored", "reason": "no subject"}), 200

company_name = subject.strip()
print("Email inbound for company: {}".format(company_name))

# Get reply addresses
reply_env = os.environ.get('REPLY_TO_EMAIL', '')
reply_addresses = [e.strip() for e in reply_env.split(',') if e.strip()]
if not reply_addresses:
    print("No REPLY_TO_EMAIL configured")
    return jsonify({"status": "error", "reason": "no reply address"}), 20

# Extract PDF attachments
attachments = []
attachment_count = int(request.form.get('attachment-count', 0))

for i in range(1, attachment_count + 1):
    att = request.files.get('attachment-{}'.format(i))
    if att and att.filename:
        ext = att.filename.rsplit('.', 1)[-1].lower()
        if ext in ('pdf', 'csv', 'txt'):
            # Save to temp file
            tmp = tempfile.NamedTemporaryFile(
                suffix='.' + ext,
                delete=False,
                dir='/tmp'
            )
            att.save(tmp.name)
            attachments.append(tmp.name)

if not attachments:
    print("No valid attachments found in email")
    # Send error reply
    send_email_with_excel(reply_addresses, company_name, None, 0)
    return jsonify({"status": "no_attachments"}), 200

# Extract text from all attachments
combined_text = ""
for fpath in attachments:
    try:
        combined_text += "\n\n=== FILE: {} ===\n".format(
            os.path.basename(fpath)) + extract_text(fpath)
    except Exception as e:
        print("Failed to read {}: {}".format(fpath, e))
    finally:
        try: os.remove(fpath)
```

```
        except: pass

if not combined_text.strip():
    print("No text extracted from attachments")
    return jsonify({"status": "no_text"}), 200

# Check Supabase for existing analysis of this company
existing_id, existing_data = supabase_get_analysis(company_name)
is_update = existing_id is not None
print("Existing analysis found: {}".format(is_update))

# Parse with Claude
try:
    new_data = parse_with_claude(combined_text, company_name)
except Exception as e:
    print("AI parsing failed: {}".format(e))
    return jsonify({"status": "parse_error"}), 200

new_data = sanitize_data(new_data)

# Run validation layers
try:
    rules_positions = run_rules_engine(combined_text)
    existing_lenders = set()
    for p in new_data.get('current_positions', []):
        lender = p['lender'].lower()
        lender_key = ' '.join(re.sub(r'\s+[a-z0-9#]{4,}\s*$', '', lender)
        existing_lenders.add(lender_key)
    for rp in rules_positions:
        lender = rp['lender'].lower()
        lender_key = ' '.join(re.sub(r'\s+[a-z0-9#]{4,}\s*$', '', lender)
        if lender_key not in existing_lenders:
            new_data.setdefault('current_positions', []).append({
                'lender': rp['lender'], 'amount': rp['amount'],
                'frequency': rp['frequency'], 'confidence': 1.0,
                'notes': rp['notes'],
                'monthly_amount': calc_monthly(rp['amount'], rp['frequenc
            })
except: pass

try:
    new_data['reconciliation_results'] = reconcile_balances(
        combined_text, new_data.get('months', []))
except: pass

try:
    new_data['continuity_flags'] = check_continuity(
```

```
            combined_text, new_data.get('months', []))
    except: pass

    try:
        new_data['verify_issues'] = verify_with_claude(combined_text, new_dat
    except: pass

    # Merge with existing if found
    if is_update and existing_data:
        new_data = merge_data(existing_data, new_data)

    # Build Excel
    try:
        excel_buf = build_excel(new_data)
        excel_bytes = excel_buf.read()
    except Exception as e:
        print("Excel build failed: {}".format(e))
        return jsonify({"status": "excel_error"}), 200

    # Save to Supabase
    try:
        supabase_save_analysis(company_name, new_data, excel_bytes)
    except Exception as e:
        print("Supabase save failed: {}".format(e))

    # Also save to local history
    try:
        save_history(company_name, new_data, excel_bytes)
    except: pass

    # Send email reply
    month_count = len(new_data.get('months', []))
    sent = send_email_with_excel(
        reply_addresses, company_name, excel_bytes,
        month_count, is_update=is_update
    )

    print("Email workflow complete. Sent: {}  Months: {}  Update: {}".format(
        sent, month_count, is_update))

    return jsonify({"status": "success", "company": company_name,
                    "months": month_count, "is_update": is_update}), 200

except Exception as e:
    print("Email inbound error: {}".format(e))
    import traceback
    traceback.print_exc()
```

```
        return jsonify({"status": "error", "message": str(e)}), 200

if __name__=='__main__':
    app.run(debug=False, host='0.0.0.0', port=int(os.environ.get('PORT', 5001)))
```